CS4104 – COMPUTER GRAPHICS AND VIRTUAL REALITY

PROJECT / CASE STUDY REPORT

TITLE OF THE SELECTED CASE STUDY

Case Study 2 – Industrial Design and Architecture

Atelier Car Designer: Collaborative 1:1 Automotive Prototyping

Submitted by

Pranav Nayak

USN: 1RVU23CSE344 | Section: C

Under the guidance of

Prof. Shyam Mohan J S

School of Computer Science and Engineering

RV University, Bengaluru

Academic Year: 2026–2027

RV UNIVERSITY

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the CS4104 – Computer Graphics and Virtual Reality case study report titled "**Atelier Car Designer: Collaborative 1:1 Automotive Prototyping**" has been prepared and submitted by **Pranav Nayak** (USN: **1RVU23CSE344**) of the School of Computer Science and Engineering, RV University, Bengaluru, during the academic year **2026–2027**. The report has been completed in accordance with the course requirements and the instructions provided.

_____

Prof. Shyam Mohan J S

Faculty Guide

ABSTRACT

Automotive design reviews have traditionally depended on full-size clay models, which take weeks to build, are costly to change and can only be inspected by people who are physically present in the studio. This report studies Case Study 2 (Industrial Design and Architecture), in which design teams in Bengaluru and Detroit review and manipulate 1:1 scale car prototypes together in a virtual environment. To ground the analysis in a working system, a prototype called *Atelier Car Designer* was built in Unity 6 with the Universal Render Pipeline. It loads a car as nine separately selectable components, places it on a turntable on a rooftop parking deck in a city at dusk, lets a designer scale, rotate and translate any component about its own centre, and shows the resulting 4 × 4 homogeneous composite matrix live. The report answers the three assignment questions: how 2D CAD drawings become 3D meshes through boundary representation and constructive solid geometry; how the composite matrix for scaling a wheel rim by 1.5, rotating it by 45° and moving it 50 mm forward is built and why the order T·R·S matters; and which clipping and hidden-surface removal algorithms keep a stereo headset inside its frame budget. Worked calculations cover the composite matrix, tessellation tolerance, the perspective projection, polygon clipping, depth-buffer precision, the 11.1 ms frame budget at 90 Hz and the Bengaluru–Detroit network delay. Measured in a development build, the prototype runs at roughly 410–460 frames per second. The report ends with a proposed headset and networking architecture, the current limitations and the future scope.

Keywords: virtual reality, homogeneous transformations, B-Rep, CSG, polygon clipping, Z-buffer, automotive design review, Unity URP

TABLE OF CONTENTS

1. Introduction **1**
1.1 Background 1
1.2 Need 1
1.3 Scope 1
2. Problem Statement and Objectives **1**
2.1 Problem Statement 1
2.2 Stakeholders 2
2.3 Objectives 2
3. Case Scenario and Requirements **2**
3.1 Scenario Summary 2
3.2 Functional Requirements 3
3.3 Non-Functional Requirements 3
4. Relevant CG/VR Concepts **3**
4.1 Solid Representations 3
4.2 Homogeneous Coordinates and Affine Transformations 4
4.3 Viewing Pipeline, Clipping and Hidden-Surface Removal 4
5. Technical Analysis and Proposed Solution **4**
5.1 Question 1 – From 2D CAD Drawings to 3D Meshes 4
5.2 Question 2 – Composite Transformation of a Wheel Rim 6
5.3 Question 3 – Clipping and Hidden-Surface Removal for Dual Displays 8
6. System Design / Workflow **11**
6.1 Architecture 11
6.2 Interaction Workflow 11
6.3 Rendering and Material Workflow 12
6.4 Proposed VR and Multi-Site Extension 12
7. Performance, Optimization and Limitations **13**
7.1 Frame Budget 13
7.2 Measured Performance of the Prototype 13
7.3 Optimisations Already in the Prototype 14
7.4 Optimisations Proposed for the Headset Build 14
7.5 Network Latency between Bengaluru and Detroit 14
7.6 Limitations 14
8. Applications and Discussion **15**
8.1 Practical Relevance and Benefits 15
8.2 Implementation Challenges 15
9. Conclusion and Future Scope **15**
References **16**

1. Introduction

1.1 Background

A new car passes through several design stages before it reaches production: concept sketches, digital surface models in CAD, and full-size physical models that the design team walks around and judges. That last stage has long relied on clay: a 1:1 model is milled from the CAD data and finished by hand so that designers can study proportions, reflections and stance at true scale. Each significant change means re-milling or re-sculpting, which costs days, and the model exists in only one studio. Virtual reality offers the same true-scale view with stereoscopic depth and head-tracked parallax, which are the two cues that make a virtual car feel like an object in the room rather than a picture on a screen [1], [2].

1.2 Need

Large manufacturers now run design centres on different continents. In the scenario for this case study, teams in Bengaluru and Detroit must look at the same prototype at the same time and change it together. A shared virtual prototype removes shipping, re-milling and travel from the loop: a change to a wheel, mirror or door becomes a numeric edit that every participant sees within a fraction of a second. For this to work, the system must represent the car correctly, transform parts precisely, and render two high-resolution views fast enough for a headset.

1.3 Scope

This report covers geometric modelling of car bodies, homogeneous transformations, clipping, hidden-surface removal and real-time performance, which are the topics named in the assignment questions. The analysis is backed by a working prototype, *Atelier Car Designer*, that runs as a Windows desktop application. Headset rendering and the multi-site network layer are designed and sized in Chapters 6 and 7 but are not yet implemented. Physics, aerodynamics and haptics are outside the scope.

2. Problem Statement and Objectives

2.1 Problem Statement

Design teams in two cities need to view and modify a 1:1 scale car prototype concurrently without a physical clay model. The system must convert CAD geometry into a form that can be rendered in real time, let a designer apply exact transformations to individual components, and draw only what each eye can see so that the headset never drops below its refresh rate.

2.2 Stakeholders

The main stakeholders are the exterior and interior **designers**, who judge proportions and surfaces at true scale; **design managers**, who approve proposals without travelling; **engineering** (packaging and manufacturing), who check clearances early; the **graphics and IT team**, who keep frame rate, latency and data security within limits; and **management**, who want to cut the cost and time spent on clay models.

2.3 Objectives

- **O1.** Represent the car as independently editable components at true 1:1 scale.
- **O2.** Let a designer scale, rotate and translate any component about its own centre, and show the exact 4 × 4 homogeneous matrix being applied.
- **O3.** Render only visible surfaces using clipping, culling and depth testing.
- **O4.** Stay within the VR frame budget of 11.1 ms per frame at 90 Hz with headroom.
- **O5.** Define an architecture that keeps Bengaluru and Detroit in the same session.

3. Case Scenario and Requirements

3.1 Scenario Summary

An automotive manufacturer replaces part of its clay-model reviews with a collaborative virtual environment. Reviewers in Bengaluru and Detroit open the same car, select parts such as wheels, doors or mirrors, change them, and discuss the result. The prototype built for this report implements the single-site review client: it shows a stylised street coupe (4.40 m × 2.37 m × 2.57 m) at 1:1 scale on a turntable on a rooftop parking deck in a city at dusk (Figure 1).



Figure 1: Main screen of the Atelier Car Designer prototype: component list (left), transform inspector with the live composite matrix (right), and finishes and view actions (bottom)

3.2 Functional Requirements

Table 1: Functional requirements and their status in the prototype

ID	Requirement	Status
FR1	Load a car model at 1:1 scale, split into named components	Done (9 parts)
FR2	Select a component by pointing at it or from a list	Done (raycast)
FR3	Scale, rotate and translate a component about its own centre	Done
FR4	Show the composite homogeneous matrix M = T·R·S live	Done
FR5	Open doors and show an exploded view of the car	Done
FR6	Change paint finish; switch between prototypes	Done (7 finishes; 1 car loaded)
FR7	Render in a stereo headset with tracked controllers	Proposed (§6.4)
FR8	Share one session between Bengaluru and Detroit	Proposed (§6.4)

3.3 Non-Functional Requirements

Table 2: Non-functional requirements

Attribute	Target
Frame rate	≥ 90 Hz per eye in a headset (frame time ≤ 11.1 ms)
Motion-to-photon latency	< 20 ms, to avoid simulator sickness [2]
Precision	Translations to 1 mm; scale to 0.01; rotation to 1°
Remote edit propagation	Visible at the other site in < 300 ms
Portability	Windows now; Android, WebGL and OpenXR headsets next
Reproducibility	Scene can be rebuilt from scripts and the model file alone

4. Relevant CG/VR Concepts

4.1 Solid Representations

A solid can be stored in several ways, and each suits a different stage of the design process [3], [4]. Table 3 compares the ones relevant to car design.

Table 3: Comparison of geometric representations

Representation	Stores	Strength	Weakness
Wireframe	Vertices and edges	Very light	Ambiguous; no faces or inside/outside
B-Rep	Faces, edges, vertices + exact surfaces (NURBS)	Exact smooth surfaces; standard in CAD	Booleans can fail near tangent faces
CSG	Tree of Boolean ops on primitives	Compact, always valid, easy to edit	Free-form body panels are hard to express
Polygon mesh	Triangles with normals and UVs	What GPUs draw; fast	Approximate; loses exact curvature

Car body panels are free-form surfaces, usually described with Non-Uniform Rational B-Splines (NURBS) [5]. A NURBS curve of degree p with control points $\mathbf{P}_i$ and weights w_i is:

$$C(u) = \frac{\sum_{i=0}^{n} N_{i,p}(u) w_i \mathbf{P}_i}{\sum_{i=0}^{n} N_{i,p}(u) w_i} \qquad (1)$$

Here $N_{i,p}$ are the B-spline basis functions. Weights let the same formula represent exact circles and conics, which is why wheel arches and rims can be modelled without approximation in CAD.

4.2 Homogeneous Coordinates and Affine Transformations

Translation cannot be written as a 3 × 3 matrix product, so graphics systems add a fourth coordinate $w = 1$ to every point [6]. Then translation, rotation and scaling are all 4 × 4 matrices and any sequence of them collapses into one matrix. Matrix multiplication is not commutative, so the order of the factors changes the result. This matters for Question 2 in Section 5.2.

4.3 Viewing Pipeline, Clipping and Hidden-Surface Removal

Each vertex passes through model, world, view and clip space before the perspective divide maps it to normalised device coordinates and then to the viewport [7]. Clipping removes the parts of primitives that fall outside the view volume, and hidden-surface removal decides which of the remaining surfaces is nearest to the eye at each pixel. Table 4 summarises the standard algorithms [6], [8], [9], [10].

Table 4: Clipping and hidden-surface removal algorithms

Algorithm	Type	Idea	Fit for VR car review
Cohen–Sutherland	Line clip	Region outcodes for trivial accept/reject	Overlay lines, guides
Cyrus–Beck / Liang–Barsky	Line clip	Parametric t against each plane	Rays, measurement lines
Sutherland–Hodgman	Polygon clip	Clip polygon against one plane at a time	Yes: frustum is convex
Weiler–Atherton	Polygon clip	Handles concave clip windows	Not needed
Back-face culling	HSR (object)	Drop faces pointing away from eye	Yes: halves work
Z-buffer	HSR (image)	Keep nearest depth per pixel	Yes: GPU standard
Painter's algorithm	HSR (object)	Draw far-to-near	Only for transparent glass
Occlusion culling	Visibility	Skip objects hidden behind others	Yes, for dense CAD models

5. Technical Analysis and Proposed Solution

5.1 Question 1 – From 2D CAD Drawings to 3D Meshes

Figure 2 shows the full path from a designer's drawings to the triangles the headset draws. Each step is explained below, together with the matching part of the prototype where there is one.

(a) Drawings to curves and surfaces

The 2D package drawings (side, plan and front views) fix the wheelbase, overhangs and character lines. These profiles are fitted with degree-3 NURBS curves, and surfaces are generated from them by **extrusion** (a profile pushed along a straight line), **revolution** (a profile spun about an axis, used for wheels and rims), **sweeping** (a profile moved along a path) and **lofting** (a surface passed through several cross-sections, used for hoods and roofs) [5]. Exterior panels are joined with curvature (G2) continuity so that reflections flow across seams without kinks. Designers call these *Class-A* surfaces.

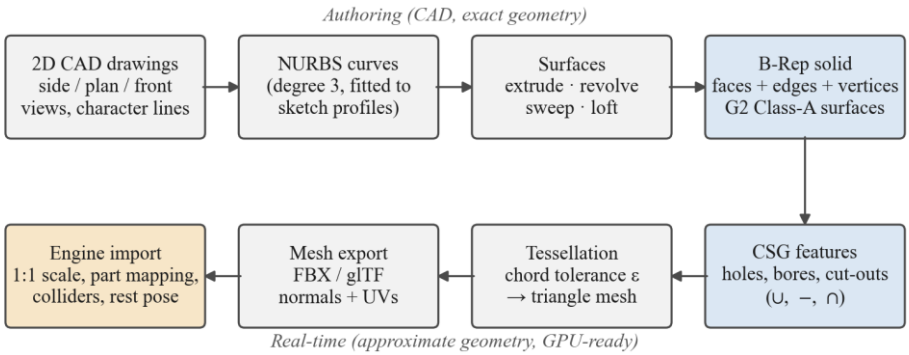


Figure 2: Pipeline from 2D CAD drawings to a real-time triangle mesh

(b) Boundary representation (B-Rep)

The surfaces are trimmed and stitched into a closed B-Rep solid: a graph of faces, edges and vertices in which each face points to its exact surface [3]. A valid closed solid without holes satisfies Euler's formula $V - E + F = 2$. For example, a box has $8 - 12 + 6 = 2$. CAD kernels use such topological checks to reject broken models before tessellation.

(c) Constructive solid geometry (CSG)

Engineered features are easier to express as Boolean operations on simple primitives. Figure 3 shows a CSG tree for a wheel rim: an outer cylinder minus the union of a hub bore and five spoke cut-outs. Modern CAD tools combine both methods. Features are recorded as a CSG-like history tree, and the result is evaluated into a B-Rep.

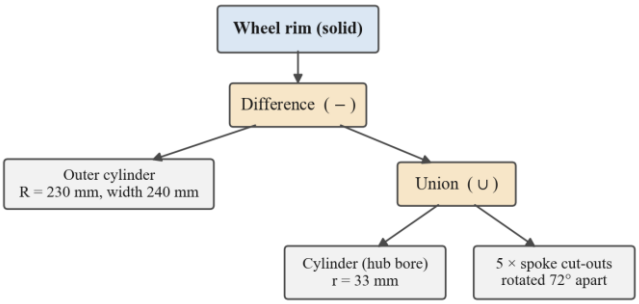


Figure 3: CSG tree for a five-spoke wheel rim

(d) Tessellation

GPUs draw triangles, so the exact surfaces are tessellated. The tessellator places vertices so that no chord deviates from the true surface by more than a tolerance ε. For a circle of radius r cut into N segments, the chord error is:

$$e = r\left(1 - \cos\frac{\pi}{N}\right) \qquad (2)$$

Worked example. For the rim edge, r = 230 mm and ε = 0.5 mm, so $\cos(\pi/N) = 1 - 0.5/230 = 0.997826$. This gives $\pi/N = 0.06595$ rad, so $N = 47.6$. Rounding up, **48 segments** keep the rim within 0.5 mm of the CAD surface. The prototype generates its display turntable the same way: `DiscMesh` uses $N = 160$ segments for $r = 3.2$ m, giving $e = 3200 \times (1 - \cos(\pi/160)) \approx 0.62$ mm.

(e) Export and engine import

The tessellated parts are exported as FBX or glTF with normals and UVs. The prototype's car comes from the BTG Low Poly Car Pack [19] (`LowPolyCarsPack.fbx`, fifteen cars). The editor script `StudioSceneBuilder` keeps one (`Car03_2`), scales the file's centimetres by 0.87 so the car is 4.40 m long, bakes the axis conversion, maps each named node to a labelled component (`WhFL` → `Tire_FL`, `DL` → `Door_L`, …), adds a mesh collider for picking and records each part's rest pose. Because the source here is already a mesh, steps (a)–(d) happen upstream in the modelling tool.

Recommendation: keep B-Rep as the master for visible Class-A surfaces, use CSG for engineered features, and send the headset only tessellated meshes with a stored link back to the CAD part ID. This keeps exact geometry in CAD and fast geometry in VR.

5.2 Question 2 – Composite Transformation of a Wheel Rim

Task. Scale the front-left wheel rim by 1.5, rotate it by 45° and move it forward by 50 mm.

Coordinate convention. Points are column vectors and $p' = M\,p$. The car's design frame has X lateral (the wheel axle), Y vertical and Z forward, with lengths in millimetres. Unity is left-handed and Y-up, and the rotation matrices take their usual numeric form. The rotation is taken about the wheel's own axle (X), so the rim spins in place. Forward means $+Z$.

Order of transformations. Scale first, then rotate, then translate:

$$\mathbf{M} = \mathbf{T}(0, 0, 50) \cdot \mathbf{R}_x(45°) \cdot \mathbf{S}(1.5) \qquad (3)$$

The individual matrices, with cos 45° = sin 45° = 0.7071, are:

$$\mathbf{S} = \begin{bmatrix} 1.5 & 0 & 0 & 0 \\ 0 & 1.5 & 0 & 0 \\ 0 & 0 & 1.5 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{R}_x = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.7071 & -0.7071 & 0 \\ 0 & 0.7071 & 0.7071 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{T} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 50 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

(4)

Step 1: R·S (each column of *R* is multiplied by 1.5):

$$\mathbf{R}_x\mathbf{S} = \begin{bmatrix} 1.5 & 0 & 0 & 0 \\ 0 & 1.0607 & -1.0607 & 0 \\ 0 & 1.0607 & 1.0607 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad (5)$$

Step 2: T·(R·S) (the translation lands in the last column):

$$\mathbf{M} = \begin{bmatrix} 1.5 & 0 & 0 & 0 \\ 0 & 1.0607 & -1.0607 & 0 \\ 0 & 1.0607 & 1.0607 & 50 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad (6)$$

Check with a point. Take a point on the rim edge, 230 mm above the wheel centre: $p = (0, 230, 0, 1)^T$. Equation (6) gives $p' = (0, 243.95, 293.95, 1)^T$. Its distance from the moved centre (0, 0, 50) is $\sqrt{(243.95^2 + 243.95^2)} = 345$ mm = 1.5 × 230 mm, and it now lies at 45° from the vertical. Scale, rotation and translation are all correct. The bottom row [0 0 0 1] keeps $w = 1$, so the transform is affine.

Why the order matters. If the translation is applied first ($M' = S{\cdot}R{\cdot}T$), the 50 mm offset is itself scaled and rotated: $R{\cdot}S{\cdot}(0, 0, 50)^T = (0, -53.03, 53.03)^T$. The wheel would move 53 mm down and 53 mm forward instead of 50 mm forward. If the rotation were instead about the vertical *Y* axis (steering the wheel), the upper-left block of *M* would become [1.0607 0 1.0607; 0 1.5 0; −1.0607 0 1.0607] with the same translation column.

Rotating about the part's own centre

The matrix above rotates and scales about the origin. To act about the rim's centre **c**, the part is moved to the origin, transformed and moved back:

$$\mathbf{M}_c = \mathbf{T}(\mathbf{c}) \cdot \mathbf{M} \cdot \mathbf{T}(-\mathbf{c}) \quad \Leftrightarrow \quad p' = \mathbf{c} + \mathbf{t} + \mathbf{R}\big(s(p - \mathbf{c})\big) \qquad (7)$$

The prototype implements this form in `CarPart.ApplyPose()` (door-swing and exploded-view offsets omitted):

```
float s = Edit.scale;                                  // 1.5
Quaternion r = frame * Edit.Rotation
             * Quaternion.Inverse(frame);            // 45° about axle
Vector3 t = frame * (Edit.translationMm * 0.001f);   // mm -> m
Vector3 position = Pivot + t + r * ((restPosition - Pivot) * s);
transform.SetLocalPositionAndRotation(position, r * restRotation);
transform.localScale = restScale * s;
```

Two design choices keep the result exact. First, the rotation is conjugated by the car's design frame ($F \cdot R \cdot F^{-1}$), so "about X" always means the car's axle even if the model was authored facing another direction. Second, every pose is rebuilt from the stored rest pose rather than multiplied onto the previous one, so repeated edits never accumulate floating-point drift. Figure 4 shows the app after the *Case Preset* button applies the task. The matrix panel reads 1.500, 1.061, −1.061 and 50.0, which matches Equation (6).



Figure 4: Case preset S1.5 · R45° · T+50 applied to the front-left wheel; the live matrix matches the hand calculation

5.3 Question 3 – Clipping and Hidden-Surface Removal for Dual Displays

A headset renders the scene twice per frame, once for each eye, from two cameras about 63 mm apart. Every triangle that is not visible therefore costs double. Single-pass instanced rendering draws each object once with one instance per eye, which roughly halves the CPU draw-call cost [11]. Visibility work is therefore done in three layers: whole objects, individual primitives and individual pixels (Figure 5).

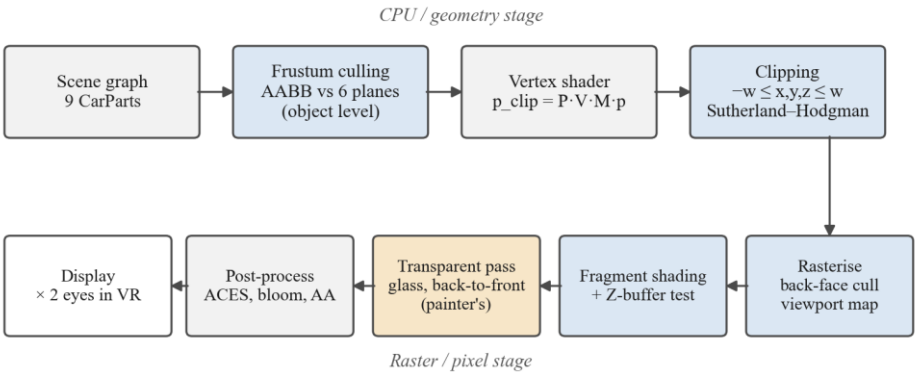


Figure 5: Rendering pipeline showing where culling, clipping and depth testing happen

(a) Object level: view-frustum culling

Each part's axis-aligned bounding box (centre **c**, half-extents **e**) is tested against the six frustum planes (**n**, d). The box is outside a plane, and can be skipped, when $\mathbf{n}\cdot\mathbf{c} + d < -\mathbf{e}\cdot|\mathbf{n}|$, where $|\mathbf{n}|$ takes each component's absolute value. The prototype builds the same kind of box for each part from its eight transformed corners (`BoundsInParent`, used for pivots and framing), and Unity culls every renderer against the camera frustum using its world-space bounds in the same way [12]. For stereo, culling is done once against a combined frustum that encloses both eyes.

(b) Primitive level: clip space and the perspective matrix

The prototype camera uses a vertical field of view of 36°, near plane n = 0.05 m, far plane f = 320 m and a 16:9 aspect ratio. With cot 18° = 3.0777, the OpenGL-style perspective matrix has entries cot(18°)/1.7778, cot(18°), $(f+n)/(n-f)$ and $2fn/(n-f)$:

$$\mathbf{P} = \begin{bmatrix} 1.7312 & 0 & 0 & 0 \\ 0 & 3.0777 & 0 & 0 \\ 0 & 0 & -1.0003 & -0.1000 \\ 0 & 0 & -1 & 0 \end{bmatrix} \qquad (8)$$

After this multiplication a vertex is inside the view volume only if $-w \le x, y, z \le w$. The frustum is convex, so the **Sutherland–Hodgman** algorithm is the right polygon clipper [8]. It clips against one plane at a time, and for each edge (S → E) it outputs E if both ends are inside, the intersection if the edge leaves, the intersection and E if it enters, and nothing if both are outside. A clipped triangle becomes a convex polygon of at most nine vertices, which is fanned back into triangles. Real GPUs use a guard band, so in practice only the near plane needs true geometric clipping.

Worked example. Clip the triangle A(2, 2), B(14, 6), C(4, 9) against the window $0 \le x, y \le 10$. Only B is outside, past the right edge $x = 10$. Its Cohen–Sutherland outcode is RIGHT, while A and C are 0000, so neither edge can be trivially accepted or rejected.

Table 5: Sutherland–Hodgman pass against the right edge x = 10

Edge	Case	Intersection	Output
A → B	in → out	t = (10 − 2)/(14 − 2) = 0.667, y = 4.67	I_1(10, 4.67)
B → C	out → in	t = (10 − 14)/(4 − 14) = 0.4, y = 7.2	I_2(10, 7.2), C
C → A	in → in	—	A

The result is the quadrilateral I_1 I_2 C A, which is split into two triangles. The same intersection follows from the Cyrus–Beck formula [9], $t = \mathbf{n}\cdot(\mathbf{P}_0 - \mathbf{A}) \,/\, \mathbf{n}\cdot(\mathbf{B} - \mathbf{A})$. With inward normal $\mathbf{n} = (-1, 0)$ and $\mathbf{P}_0 = (10, 0)$, $t = -8/-12 = 0.667$.

(c) Pixel level: back-face culling and the Z-buffer

For a closed opaque body, a face whose normal points away from the eye **E** is never visible. It is culled when $\mathbf{N}\cdot(\mathbf{P} - \mathbf{E}) \geq 0$, which removes about half of the car's triangles before rasterisation. The remaining overlaps are resolved by the **Z-buffer** [10]: each pixel keeps the nearest depth drawn so far, and a new fragment is written only if it is nearer. The Z-buffer needs no sorting, works for any pose (open doors, exploded parts) and runs in parallel on the GPU. Its precision depends on the near plane [13]:

$$\Delta z \approx \frac{z^2(f-n)}{f\,n\,2^b} \qquad (9)$$

With a 24-bit buffer ($b = 24$), *$n = 0.05$ m and $f = 320$ m, Equation (9) gives $\Delta z \approx 0.03$* mm at 5 m and 0.48 mm at 20 m. That is well below the gap between body panels, so z-fighting is not a risk at car scale. Moving the near plane to 0.01 m would make it five times worse. For larger scenes, a floating-point reversed-Z buffer spreads precision more evenly [13].

(d) Transparent glass and occlusion culling

Glass cannot use the Z-buffer alone because blended colours depend on draw order. The prototype's glass material is drawn after all opaque parts, sorted back-to-front (the painter's algorithm), with depth testing on and depth writing off. For full-detail CAD models with millions of triangles, **occlusion culling** should be added so that seats, dashboard and engine parts are skipped when hidden behind closed body panels [14]. At 1.9K triangles for the car (20K for the scene) this is not yet needed.

Answer to Question 3. For the headset's dual displays, the recommended pipeline is: frustum culling of each part's bounding box (once, against the combined stereo frustum); occlusion culling for dense interiors; Sutherland–Hodgman clipping in homogeneous clip space; back-face culling; the Z-buffer for opaque surfaces; and back-to-front sorting only for transparent glass. All of these except occlusion culling already happen in the prototype, with clip-space clipping performed by the GPU's fixed-function hardware.

6. System Design / Workflow

6.1 Architecture

The prototype is organised in four layers (Figure 6). Input components turn mouse and touch gestures into camera motion and taps. The interaction layer turns taps into selections and slider changes into edits. The model layer stores and applies the edits, and URP renders the result.

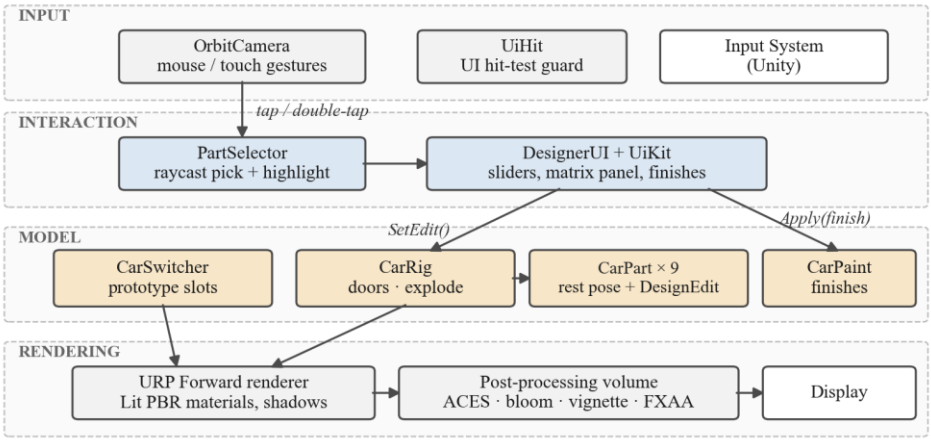


Figure 6: Software architecture of the Atelier Car Designer prototype

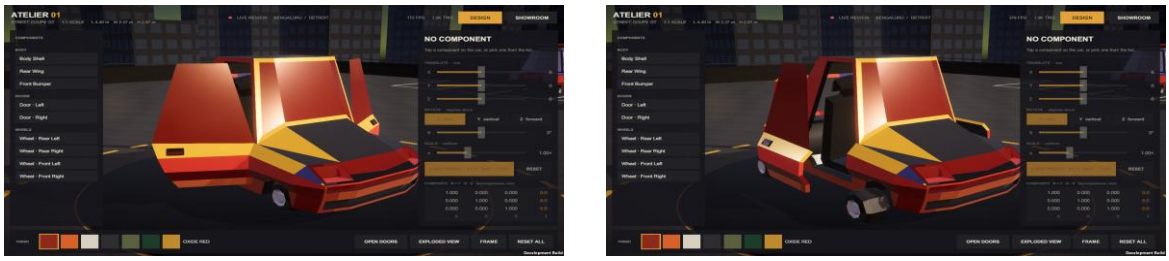
Table 6: Main components and responsibilities

Component	Responsibility
CarRig	Owns the parts; holds the car's design frame; animates doors (62° hinge swing) and exploded view
CarPart / DesignEdit	Stores rest pose and the edit (translation in mm, axis, angle, uniform scale); applies $p' = c + t + R(s(p - c))$
PartSelector	Casts a ray from the tap point against mesh colliders; draws an additive rim highlight on the selected part
OrbitCamera	Turntable camera: orbit, pinch zoom, pan, pitch limits 2°–82°, exponential smoothing
CarPaint	Swaps body finishes by recolouring the palette texture texel by texel
DesignerUI / UiKit	Builds the whole interface in code: part list, sliders, live matrix panel, finishes
StudioSceneBuilder	Editor script that rebuilds the whole scene (car and city garage) from the model file; CarSwitcher holds the car

6.2 Interaction Workflow

A tap is first checked against the UI (`UiHit`) so that gestures on panels never reach the 3D view. Otherwise `PartSelector` casts a ray from the camera through the tap point, and the first mesh collider hit identifies the `CarPart`. Moving a slider creates a new `DesignEdit`, and `CarRig.SetEdit()` rebuilds that part's pose and refreshes the matrix panel. Door opening rotates each door about a hinge at its front edge, and any side glass and mirrors are

attached so that they swing with it. Both animations are eased with the smoothstep function $s(t) = t^2(3 - 2t)$ (Figure 7).



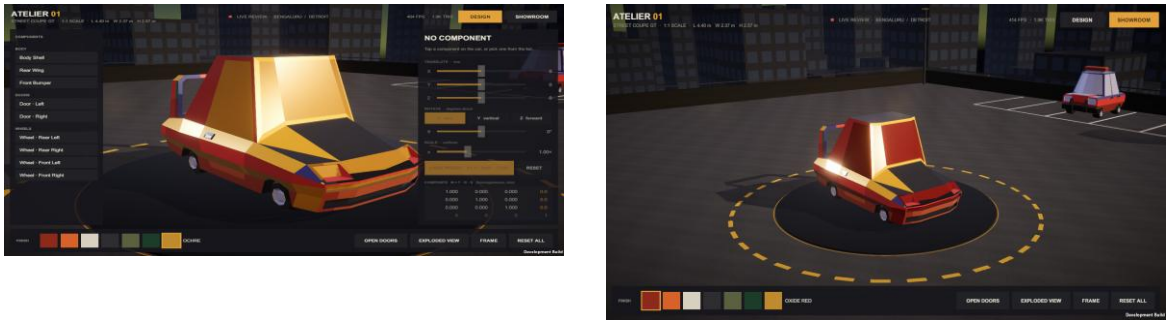
(a) Doors open *(b) Exploded view*

Figure 7: Door hinge animation and exploded view, both driven by the same pose function

6.3 Rendering and Material Workflow

The background, a rooftop parking deck in a city at dusk with parked cars and a skyline of emissive windows, is generated from code by `CityGarageEnvironment`. A sunset sky, distance fog, a trilight ambient term and a 32 × 32 city reflection cubemap set the mood, and a low evening-sun key light with soft shadows plus fill and rim lights light the car. Materials use URP's Lit shader (paint smoothness 0.82). Post-processing applies ACES tonemapping, bloom, vignette and FXAA [15]. Paint finishes are changed by recolouring only the texels near the body's key colour (RGB distance < 0.3), keeping each texel's shading:

$$c_{\text{new}} = c_{\text{paint}} \times \text{clamp}\left(\frac{\max(c_{\text{texel}})}{\max(c_{\text{key}})}, 0.6, 1.25\right) \qquad (10)$$



(a) Ochre finish on the street coupe *(b) Showroom mode in the city garage*

Figure 8: Paint finish change and the city garage background

6.4 Proposed VR and Multi-Site Extension

The prototype's logic carries over to a headset almost unchanged. Only input and output change (Figure 9). An OpenXR build [16] renders each eye with single-pass instancing, and the tap ray from `ScreenPointToRay` is replaced by the controller's pointing ray, which goes into the same `Physics.Raycast`. For collaboration, each site keeps a full local copy of the car and renders locally. Only edits travel over the network. A session server (for example, Unity Netcode for GameObjects [17]) holds the authoritative `DesignEdit` for every part,

grants a per-part edit lock so that two people cannot drag the same wheel at once, and relays voice.

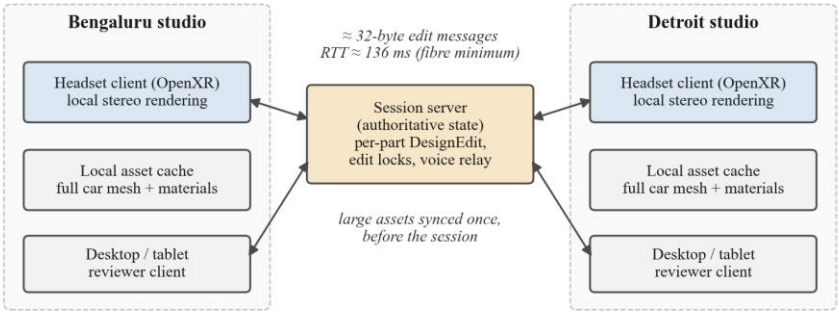


Figure 9: Proposed Bengaluru–Detroit collaboration architecture

7. Performance, Optimization and Limitations

7.1 Frame Budget

The time available for one frame is the reciprocal of the refresh rate:

$$t_{\text{frame}} = \frac{1}{f} = \frac{1}{90\ \text{Hz}} = 11.11\ \text{ms} \qquad (11)$$

At 72 Hz the budget is 13.89 ms, and at 120 Hz it is 8.33 ms. Within that time the system must draw two views. A current standalone headset shows 2064 × 2208 pixels per eye [18], which is 9.11 million pixels per frame or 820 million pixels per second at 90 Hz. That is 6.3 times the pixels of the 1600 × 900 window used for the measurements below. If a frame takes longer than the budget, the headset repeats or reprojects the old image, and the user sees judder. That is why the budget is a hard limit, not an average.

7.2 Measured Performance of the Prototype

Table 7: Frame rate averaged over 4 s, vsync off (development build, 1600 × 900)

Scenario	Triangles	FPS	Frame time
Idle, street coupe	1.9K	437	2.29 ms
Case preset applied	1.9K	438	2.29 ms
Doors open	1.9K	461	2.17 ms
Exploded view	1.9K	421	2.38 ms
Ochre finish	1.9K	434	2.30 ms
Showroom, wide city view	1.9K	412	2.43 ms
Whole scene (car + city garage)	20.3K	412–461	2.17–2.43 ms

The worst case, the showroom view, takes 2.43 ms. As a pessimistic bound, if cost grew in direct proportion to pixel count on the same GPU, stereo headset rendering would need about 2.43 × 6.3 ≈ 15.3 ms, which is over the 11.1 ms budget. A mobile headset GPU is also weaker, so the optimisations in Section 7.4 are still required.

7.3 Optimisations Already in the Prototype

- **Work only on change:** `CarRig` re-applies poses only while an edit or animation runs, and the paint texture is rewritten once per finish change, so an idle car costs almost no CPU time.
- **No drift:** poses are always rebuilt from the rest pose, so long sessions stay exact.
- **Cheap lighting and geometry:** only the key light casts shadows, the city is static and batched, the cubemap is 32 × 32 per face, and the car is 1.9K triangles.

7.4 Optimisations Proposed for the Headset Build

- **Single-pass instanced stereo** to draw both eyes in one pass and halve draw-call overhead [11].
- **MSAA 4× instead of FXAA:** post-process AA blurs fine detail and adds a full-screen pass, while MSAA is nearly free on tile-based mobile GPUs and stays stable in stereo [7].
- **Fixed foveated rendering:** shade the periphery of each eye at lower resolution.
- **Level of detail (LOD)** for CAD-density meshes, and **occlusion culling** for the interior [14].
- **GPU instancing** for the four identical wheels, and baked lighting for the static city deck.

7.5 Network Latency between Bengaluru and Detroit

The great-circle distance between the two cities is about 13,550 km. Light in optical fibre travels at about 2×10^8 m/s, so the one-way delay is at least 13,550 km ÷ 200,000 km/s = 67.8 ms and the round trip is at least 135.5 ms. Real routes are longer, so 200–300 ms is realistic. Streaming rendered frames from one site to the other could never meet the 20 ms motion-to-photon target. This is why the design renders locally and synchronises only state. One edit message holds a part ID (2 bytes), translation (12 bytes), angle and scale (8 bytes), axis (1 byte) and a sequence number (8 bytes), about 32 bytes in total. At 30 updates per second while a slider is dragged, that is under 1 kB/s per user.

7.6 Limitations

The prototype has no headset build or networking yet; the "Live Review · Bengaluru / Detroit" banner marks the intended use. The car is a low-poly mesh rather than real CAD data, so B-Rep import and tessellation happen outside the app. Only uniform scaling is supported, FXAA is used rather than MSAA, and performance was measured on a development machine rather than on headset hardware.

8. Applications and Discussion

8.1 Practical Relevance and Benefits

The same tool supports several stages of a car programme: exterior design reviews (wheel size and position, which is exactly the Question 2 task), colour-and-trim selection (the finish palette), packaging checks (opening the doors to inspect the cabin), engineering and training (the exploded view), and marketing (the auto-rotating showroom mode). A digital review removes the days needed to re-mill clay, and both studios see the same model at the same moment. Because the composite matrix is shown with each edit, a reviewer can state precisely what changed, for example "rim scaled 1.5×, rotated 45° about the axle, moved 50 mm forward", and engineering can reproduce it in CAD.

8.2 Implementation Challenges

Converting CAD data is the hardest step. Production models contain millions of faces, and tessellation loses parametric information, so LODs and a link back to CAD part IDs are needed. Paint realism needs clear-coat shading. Comfort requires a steady 90 Hz or better. Unreleased designs must be protected in transit and at rest. These are trade-offs: higher fidelity costs frame time and stronger security adds latency, so each must be balanced against the frame budget derived in Chapter 7.

9. Conclusion and Future Scope

This case study showed how core computer graphics techniques make a collaborative 1:1 car review possible. CAD surfaces are built as B-Rep solids with CSG features and tessellated within a chosen tolerance; 48 segments keep a 230 mm rim within 0.5 mm. The required wheel-rim edit is the single matrix $M = T \cdot R \cdot S$, verified by hand and by the prototype's live matrix display. The order of the factors visibly changes the result. Visibility is handled in layers: frustum culling, Sutherland–Hodgman clipping in clip space, back-face culling, a Z-buffer with ample precision at car scale, and back-to-front sorting for glass. The prototype runs at 412–461 FPS, and Chapter 7 shows which optimisations are still needed for 90 Hz in stereo.

Future scope: an OpenXR headset build with controller-based selection; the networked session server with avatars, voice and per-part locks; a CAD import path with automatic LODs; a clear-coat car paint shader; measurement and annotation tools; non-uniform scaling; and an AR mode that overlays digital parts on a physical buck.

References

[1] I. E. Sutherland, "A head-mounted three dimensional display," in Proc. AFIPS Fall Joint Computer Conf., San Francisco, CA, USA, 1968, pp. 757–764.

[2] S. M. LaValle, Virtual Reality. Cambridge, U.K.: Cambridge Univ. Press, 2023.

[3] A. A. G. Requicha, "Representations for rigid solids: Theory, methods, and systems," ACM Comput. Surv., vol. 12, no. 4, pp. 437–464, Dec. 1980.

[4] J. F. Hughes, A. van Dam, M. McGuire, D. F. Sklar, J. D. Foley, S. K. Feiner, and K. Akeley, Computer Graphics: Principles and Practice, 3rd ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2013.

[5] L. Piegl and W. Tiller, The NURBS Book, 2nd ed. Berlin, Germany: Springer, 1997.

[6] D. Hearn, M. P. Baker, and W. R. Carithers, Computer Graphics with OpenGL, 4th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2011.

[7] T. Akenine-Möller, E. Haines, N. Hoffman, A. Pesce, M. Iwanicki, and S. Hillaire, Real-Time Rendering, 4th ed. Boca Raton, FL, USA: CRC Press, 2018.

[8] I. E. Sutherland and G. W. Hodgman, "Reentrant polygon clipping," Commun. ACM, vol. 17, no. 1, pp. 32–42, Jan. 1974.

[9] M. Cyrus and J. Beck, "Generalized two- and three-dimensional clipping," Comput. Graph., vol. 3, no. 1, pp. 23–28, 1978.

[10] E. Catmull, "A subdivision algorithm for computer display of curved surfaces," Ph.D. dissertation, Dept. Comput. Sci., Univ. of Utah, Salt Lake City, UT, USA, 1974.

[11] Unity Technologies, "Single-pass instanced rendering," Unity 6 Manual. [Online]. Available: https://docs.unity3d.com/6000.0/Documentation/Manual/SinglePassInstancing.html. Accessed: Nov. 5, 2026.

[12] Unity Technologies, "Universal Render Pipeline," Unity 6 Manual. [Online]. Available: https://docs.unity3d.com/6000.0/Documentation/Manual/urp/urp-introduction.html. Accessed: Nov. 5, 2026.

[13] N. Reed, "Depth precision visualized," NVIDIA Developer Blog, Jul. 2015. [Online]. Available: https://developer.nvidia.com/content/depth-precision-visualized

[14] Unity Technologies, "Occlusion culling," Unity 6 Manual. [Online]. Available: https://docs.unity3d.com/6000.0/Documentation/Manual/OcclusionCulling.html. Accessed: Nov. 5, 2026.

[15] T. Lottes, "FXAA," NVIDIA Corp., Santa Clara, CA, USA, White Paper, Feb. 2009.

[16] The Khronos Group, "The OpenXR specification," ver. 1.1. [Online]. Available: https://registry.khronos.org/OpenXR/. Accessed: Nov. 5, 2026.

[17] Unity Technologies, "Netcode for GameObjects," Unity Multiplayer Documentation. [Online]. Available: https://docs-multiplayer.unity3d.com/netcode/current/about/. Accessed: Nov. 5, 2026.

[18] Meta Platforms, "Meta Quest 3: Tech specs." [Online]. Available: https://www.meta.com/quest/quest-3/. Accessed: Nov. 5, 2026.

[19] BetelgeuseMan, "BTG Low Poly Car Pack," CC BY 4.0. https://betelgeuseman.itch.io/lpc-pack